Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

Implementei a sincronização do embarque no metrô, onde cada carro e cada passageiro é uma thread. Fiz as 4 funções (station_init, station_wait_for_car, station_on_board e station_load_car usando apenas um mutex e duas variáveis de condição, garantindo que o passageiro só embarque quando tiver carro com lugar e que o carro só saia quando todos sentarem. Avaliei tudo com o teste que Marcelo passou.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Struct station e station_init	OK	Ver sync_car.h e sync_car.c
station_wait_for_car só libera com carro e lugar livre	OK	Rodar ./sync_car
station_on_board avisa o carro	OK	O teste chama on_board pra cada passageiro liberado
station_load_car só retorna com todos sentados	OK	Cada carro mostra “(expected N)” igual ao número que embarcou
Carro parte cheio ou quando ninguém mais espera	OK	Último carro parte com menos gente que os assentos
Carro sem passageiros ou com 0 lugares parte na hora	OK	Primeiros testes do runner, com alarme de 1 e 2 segundos
Só funções pthread permitidas, sem semáforo	OK	Procurar no sync_car.c: só mutex e condição
Um único lock por station	OK	A struct só tem um pthread_mutex_t
Passageiros embarcando ao mesmo tempo	OK	O runner libera vários antes de chamar on_board

Sem busy-wait	OK	Toda espera é pthread_cond_wait dentro de while
Makefile gerando sync_car e limpando	OK	make clean e make
evidencias.log	OK	

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. sync_car.h - guarda a struct station e as assinaturas das funções.
2. sync_car.c - tem as funções station_init, station_wait_for_car, station_on_board e station_load_car.
3. sync_car-runner.c - teste e main.
4. Makefile - compila e limpa tudo.
5. evidencias.log - registro do terminal com os testes que fiz.
6. README.md - explica os arquivos e como compilar e rodar.

Decisões técnicas:

1. Usar 3 contadores, pois um passageiro pode já ter sido liberado e ainda não ter sentado. Assim o carro só sai quando ninguém está mais a caminho.
2. Reservar o assento dentro do lock, quando o passageiro sai do wait_for_car, pra dois passageiros não pegarem o mesmo último lugar.
3. Broadcast pra os passageiros e signal pra o carro, pois cabem vários passageiros, mas só existe um carro por vez.

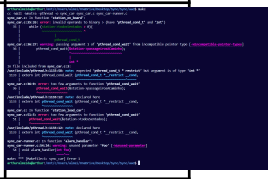
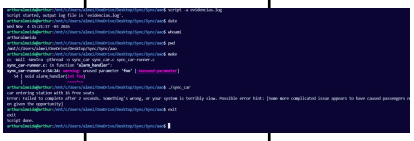
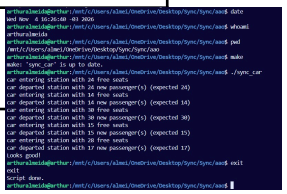
5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

.

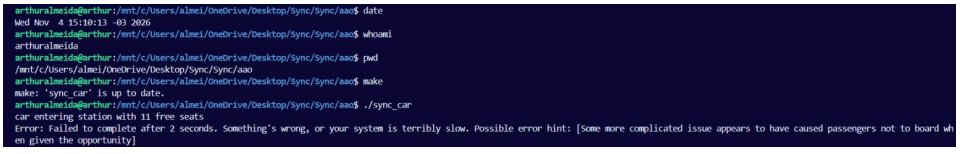
Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Implementar tudo e testar apenas no final	Implementei todas as funções antes de testar e testei tudo de uma vez	Não houve

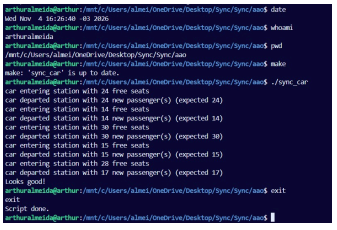
5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Fazer o teste geral	Falha	Comparei variável de condição com um inteiro e faltou o mutex	04/11 às 14:44	
2	S1	Corrigir erros de compilação	Sucesso		04/11 às 15h	
3	S1	Rodar o teste do professor	Falha	A load_car dormia sem conferir nada e sem passageiros ninguém acordava o carro	04/11 às 15h	
4	S1	Colocar o while com a condição de partida em station_load_car	Falha	A wait_for_car saía sem soltar o mutex e travava todo mundo	04/11 às 15:10h	
5	S1	Colocar unlock e trocar if por while na wait_for_car	Falha	A on_board esperava passageirosACaminho zerar, ou seja, esperava ela mesma	04/11 às 15:21	
6	S1	Testar	Sucesso		04/11 às 16:26	

6. Evidências

6.2 Prints





A wait_for_car não fazia unlock no fim e travava todas as threads. Consegui consertar o erro, depois surgiu um outro erro mas no final deu tudo certo.

7. Uso de IA

O prompt inicial foi “preciso fazer essa implementação e vou começar do 0, me ajude a fazer

me ensinando a aprender e me fazendo pensar sozinho”. Utilizei a IA para entender o que precisava ser feito e para entender meus erros.

8. Reflexão Final (5–8 linhas)

Minha decisão mais importante foi o contador passageirosACaminho, que impede o carro de sair com passageiros ainda indo sentar. Os meus erros mais demorados foram confundir variável de condição com contador e não colocar o unlock na wait_for_car, o que travou tudo. Com mais tempo faria testes próprios e faria mais tratamentos de erro para deixar o código mais completo.

9. Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Com mais duas horas eu começaria corrigindo o caso de um carro chegar com número negativo de assentos. Também rodaria o programa mais vezes e usando ferramentas que encontram problemas entre as threads. Faria mais testes diferentes para ter a certeza de que a implementação funcionaria em qualquer cenário, não só as do teste de Marcelo.